Read One Row

You already know how to filter rows to select using `.where()`.

And you saw how when executing a `select()` it normally returns an **iterable** object.

Or you can call `results.all()` to get a **list** of all the rows right away, instead of an iterable.

But in many cases you really just want to read a **single row**, and having to deal with an iterable or a list is not as convenient.

Let's see the utilities to read a single row.

Continue From Previous Code

We'll continue with the same examples we have been using in the previous chapters to create and select data and we'll keep updating them.

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

If you already executed the previous examples and have a database with data, **remove the database file** before running each example, that way you won't have duplicate data and you will be able to get the same results.

Read the First Row

We have been iterating over the rows in a `result` object like:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        for hero in results:
            print(hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

But let's say that we are not interested in all the rows, just the **first** one.

We can call the `.first()` method on the `results` object to get the first row:

Python 3.10+

```python
# Code above omitted 👆
```

```python
def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        hero = results.first()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        hero = results.first()
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

This will return the first object in the `results` (if there was any).

That way, we don't have to deal with an iterable or a list.

> **Tip**
>
> Notice that `.first()` is a method of the `results` object, not of the `select()` statement.

Although this query would find two rows, by using `.first()` we get only the first row.

If we run it in the command line it would output:

```bash
```

First or `None`

It would be possible that the SQL query doesn't find any row.

In that case, `.first()` will return `None`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age < 25)
        results = session.exec(statement)
        hero = results.first()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age < 25)
        results = session.exec(statement)
        hero = results.first()
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

In this case, as there's no hero with an age less than 25, `.first()` will return `None`.

When we run it in the command line it will output:

bash

Exactly One

There might be cases where we want to ensure that there's exactly **one** row matching the query.

And if there was more than one, it would mean that there's an error in the system, and we should terminate with an error.

In that case, instead of `.first()` we can use `.one()`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.name == "Deadpond")
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Here we know that there's only one `"Deadpond"`, and there shouldn't be any more than one.

If we run it once, it will output:

```bash
```

But if we run it again, as it will create and insert all the heroes in the database again, they will be duplicated, and there will be more than one `"Deadpond"`. 😱

So, running it again, without first deleting the file `database.db` will output:

```bash
```

Exactly One with More Data

Of course, even if we don't duplicate the data, we could get the same error if we send a query that finds more than one row and expect exactly one with `.one()`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age <= 35)
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

That would find 2 rows, and would end up with the same error.

Exactly One with No Data

And also, if we get no rows at all with `.one()`, it will also raise an error:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age < 25)
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.age < 25)
        results = session.exec(statement)
        hero = results.one()
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

In this case, as there are no heroes with an age less than 25, `.one()` will raise an error.

This is what we would get if we run it in the command line:

```bash
```

Compact Version

Of course, with `.first()` and `.one()` you would also probably write all that in a more compact form most of the time, all in a single line (or at least a single Python statement):

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        hero = session.exec(select(Hero).where(Hero.name == "Deadpond")).one()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        hero = session.exec(select(Hero).where(Hero.name == "Deadpond")).one()
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

That would result in the same as some examples above.

Select by Id with `.where()`

In many cases you might want to select a single row by its Id column with the **primary key**.

You could do it the same way we have been doing with a `.where()` and then getting the first item with `.first()`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.id == 1)
        results = session.exec(statement)
        hero = results.first()
        print("Hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).where(Hero.id == 1)
        results = session.exec(statement)
        hero = results.first()
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

That would work correctly, as expected. But there's a shorter version. 👇

Select by Id with `.get()`

As selecting a single row by its Id column with the **primary key** is a common operation, there's a shortcut for it:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        hero = session.get(Hero, 1)
        print("Hero:", hero)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        hero = session.get(Hero, 1)
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

`session.get(Hero, 1)` is an equivalent to creating a `select()`, then filtering by Id using `.where()`, and then getting the first item with `.first()`.

If you run it, it will output:

```bash
```

Select by Id with `.get()` with No Data

`.get()` behaves similar to `.first()`, if there's no data it will simply return `None` (instead of raising an error):

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        hero = session.get(Hero, 9001)
        print("Hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    hero_1 = Hero(name="Deadpond", secret_name="Dive Wilson")
    hero_2 = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
    hero_3 = Hero(name="Rusty-Man", secret_name="Tommy Sharp", age=48)
    hero_4 = Hero(name="Tarantula", secret_name="Natalia Roman-on", age=32)
    hero_5 = Hero(name="Black Lion", secret_name="Trevor Challa", age=35)
    hero_6 = Hero(name="Dr. Weird", secret_name="Steve Weird", age=36)
    hero_7 = Hero(name="Captain North America", secret_name="Esteban Rogelios", age=93)

    with Session(engine) as session:
        session.add(hero_1)
        session.add(hero_2)
        session.add(hero_3)
        session.add(hero_4)
        session.add(hero_5)
        session.add(hero_6)
        session.add(hero_7)

        session.commit()


def select_heroes():
    with Session(engine) as session:
        hero = session.get(Hero, 9001)
        print("Hero:", hero)


def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Running that will output:

```bash
```

Recap

As querying the SQL database for a single row is a common operation, you now have several tools to do it in a short and simple way. 🎉